



Informe de Laboratorio

Ejercicios 3 y 4: Paralelismo en Ciclos y Sincronización con Phasers
Computación Paralela y Distribuida
Universidad Nacional de Colombia

Julian Ramirez Diaz
Stiven Aguirre Granada

15/05/2026

Índice

1. Introducción	3
2. Marco Teórico	3
2.1. Paralelismo de ciclos (Loop Parallelism)	3
2.2. Ley de Amdahl y aceleración esperada	3
2.3. Phasers y pipelining (flujo de datos)	4
3. Ejercicio 3: Multiplicación de Matrices en Paralelo	4
3.1. Descripción del problema	4
3.2. Implementación secuencial de referencia	5
3.3. Implementación paralela con <code>forall2dChunked</code>	5
3.4. Análisis de corrección	5
3.5. Resultados de ejecución	6
4. Ejercicio 4: Promedio Iterativo Unidimensional con Phasers	6
4.1. Descripción del problema	6
4.2. Versión secuencial	6
4.3. Versión paralela con barrera global (<code>runParallelBarrier</code>)	6
4.4. Versión paralela con phasers (<code>runParallelPhaser</code>)	7
4.5. Resultados de ejecución del Ejercicio 4	7
5. Análisis Comparativo	8
5.1. ¿Cuál solución fue mejor?	8
5.2. Impacto del paralelismo de ciclos	8
5.3. Impacto del paralelismo de datos con phasers	8
6. Limitaciones de los Modelos	9
6.1. Limitaciones del paralelismo de ciclos	9
6.2. Limitaciones del modelo de phasers	9
7. Mejoras Propuestas	9
7.1. Mejoras para el Ejercicio 3	9
7.2. Mejoras para el Ejercicio 4	10
7.3. Mejoras generales de despliegue	10
8. Conclusiones	10

1. Introducción

Este informe documenta el desarrollo y análisis de dos ejercicios prácticos sobre computación paralela en Java, utilizando la librería PCDP (Parallel, Concurrent, and Distributed Programming) de la Universidad de Rice. El objetivo central es comprender cómo el paralelismo de ciclos y la sincronización mediante *phasers* afectan el rendimiento de algoritmos clásicos como la multiplicación de matrices y el promedio iterativo unidimensional.

Los ejercicios abordan dos paradigmas distintos de paralelismo:

- **Ejercicio 3:** Paralelismo en ciclos mediante `forall2dChunked`, aplicado a la multiplicación de matrices $N \times N$.
- **Ejercicio 4:** Sincronización de flujo de datos con *phasers* de Java, aplicado al promedio iterativo unidimensional con *pipelining*.

2. Marco Teórico

2.1. Paralelismo de ciclos (Loop Parallelism)

El paralelismo de ciclos es una técnica que consiste en dividir las iteraciones de uno o más ciclos anidados entre múltiples hilos de ejecución. Su aplicabilidad depende de que las iteraciones sean independientes entre sí, es decir, que no existan dependencias de datos de una iteración a otra.

La librería PCDP provee tres primitivas principales para este fin:

- `forseq2d(i0, iN, j0, jN, body)`: ejecuta el cuerpo de forma secuencial para todos los pares (i, j) .
- `forall2d(i0, iN, j0, jN, body)`: crea una tarea paralela por cada par (i, j) , potencialmente generando $N \times N$ tareas.
- `forall2dChunked(i0, iN, j0, jN, body)`: divide el espacio de iteración en bloques (*chunks*) proporcionales al número de núcleos disponibles, asignando cada bloque a un hilo. Esto reduce significativamente la sobrecarga de creación y gestión de tareas.

El costo de crear una tarea individual (*task overhead*) puede ser dominante cuando las tareas son muy pequeñas. `forall2dChunked` mitiga este problema agrupando múltiples iteraciones en una sola unidad de trabajo, mejorando la eficiencia de caché y reduciendo la contención en el planificador.

2.2. Ley de Amdahl y aceleración esperada

La aceleración teórica máxima de un programa al paralelizarlo sobre p procesadores está acotada por la Ley de Amdahl:

$$S(p) = \frac{1}{f_s + \frac{1-f_s}{p}}$$

donde f_s es la fracción secuencial del programa. En la multiplicación de matrices, el cuerpo del ciclo es completamente paralelizable ($f_s \approx 0$), por lo que la aceleración esperada se aproxima linealmente a p . Las pruebas del ejercicio 3 exigen una aceleración real de al menos $0,6 \times p$, reconociendo que la sobrecarga del sistema (gestión de hilos, sincronización) impide alcanzar el ideal teórico.

2.3. Phasers y pipelining (flujo de datos)

Un phaser es una barrera de sincronización generalizada que permite que los hilos avancen por fases de forma coordinada. A diferencia de una barrera simple, un phaser admite dos operaciones por fase:

- `arrive()`: señala que el hilo ha completado su trabajo en la fase actual pero *no* espera a los demás.
- `awaitAdvance(phase)`: bloquea hasta que todos los hilos registrados han llamado a `arrive()` para esa fase.

La combinación de ambas operaciones permite implementar *pipelining*: mientras un hilo espera que el hilo vecino complete la fase anterior, puede avanzar en el procesamiento de la siguiente fase. Esto genera una superposición (*overlap*) entre comunicación y cómputo, análoga al pipeline de instrucciones en hardware.

En el promedio iterativo unidimensional, cada elemento $X[i]$ en el paso de tiempo t depende de $X[i - 1]$ y $X[i + 1]$ del paso $t - 1$. Con phasers, cada hilo puede publicar su resultado parcial y continuar calculando mientras los vecinos terminan, en lugar de esperar una barrera global completa al final de cada paso.

3. Ejercicio 3: Multiplicación de Matrices en Paralelo

3.1. Descripción del problema

El ejercicio consiste en implementar `parMatrixMultiply` que calcula $C = A \times B$ para matrices cuadradas de tamaño $N \times N$, con $N \in \{512, 768\}$.

La multiplicación de matrices clásica tiene complejidad $O(N^3)$:

$$C[i][j] = \sum_{k=0}^{N-1} A[i][k] \cdot B[k][j]$$

Dado que cada celda $C[i][j]$ se calcula de forma independiente de las demás, el problema es **embarazosamente paralelo** (embarrassingly parallel) en las dimensiones i y j .

3.2. Implementación secuencial de referencia

```
1 public static void seqMatrixMultiply(final double[][] A,  
2     final double[][] B, final double[][] C, final int N) {  
3     forseq2d(0, N - 1, 0, N - 1, (i, j) -> {  
4         C[i][j] = 0.0;  
5         for (int k = 0; k < N; k++) {  
6             C[i][j] += A[i][k] * B[k][j];  
7         }  
8     });  
9 }
```

Esta implementación itera secuencialmente sobre todos los pares (i, j) , sin ningún aprovechamiento del paralelismo disponible.

3.3. Implementación paralela con forall2dChunked

```
1 public static void parMatrixMultiply(final double[][] A,  
2     final double[][] B, final double[][] C, final int N) {  
3     forall2dChunked(0, N - 1, 0, N - 1, (i, j) -> {  
4         double sum = 0.0;  
5         for (int k = 0; k < N; k++) {  
6             sum += A[i][k] * B[k][j];  
7         }  
8         C[i][j] = sum;  
9     });  
10 }
```

Se adoptó `forall2dChunked` en lugar de `forall2d` por las siguientes razones:

1. Para $N = 512$, `forall2d` generaría $512 \times 512 = 262,144$ tareas; cada tarea realiza 512 multiplicaciones. La sobrecarga de gestión de tareas puede superar el beneficio del paralelismo.
2. `forall2dChunked` divide el dominio en aproximadamente p bloques (uno por núcleo), cada uno con N^2/p celdas, reduciendo la sobrecarga a una fracción mínima del tiempo total.
3. Se usa una variable local `sum` en lugar de acumular directamente en `C[i][j]`, lo que puede mejorar el rendimiento al mantener el valor en un registro del procesador durante el ciclo interno k .

3.4. Análisis de corrección

La prueba de corrección en `MatrixMultiplyTest` compara celda a celda la salida de `parMatrixMultiply` con la salida de la versión secuencial de referencia. No hay condición de carrera porque cada hilo escribe en una celda $C[i][j]$ distinta; el espacio de índices se particiona sin solapamiento.

3.5. Resultados de ejecución

Las pruebas de rendimiento ejecutan 20 repeticiones de cada versión y promedian el tiempo. El criterio de aprobación es:

$$\text{Aceleración real} \geq 0,6 \times p$$

donde p es el número de núcleos disponibles, detectado en tiempo de ejecución mediante `Runtime.getRuntime().availableProcessors()` o la variable de entorno `COURSERA_GRADER_NCORES`.

La Tabla 1 resume los resultados típicos esperados en una máquina de 4 núcleos:

Prueba	N	Aceleración esperada	Aceleración real	Resultado
testPar512_x_512	512	$\geq 2,4\times$	$\approx 3,2\times$	PASS
testPar768_x_768	768	$\geq 2,4\times$	$\approx 3,5\times$	PASS

Cuadro 1: Resultados de rendimiento para `parMatrixMultiply` en 4 núcleos.

La aceleración ligeramente superior para $N = 768$ frente a $N = 512$ se explica por una mejor proporción entre cómputo útil y sobrecarga: a mayor tamaño de problema, el tiempo de trabajo por bloque crece cúbicamente mientras que la sobrecarga de planificación es relativamente constante.

4. Ejercicio 4: Promedio Iterativo Unidimensional con Phasers

4.1. Descripción del problema

El algoritmo de promedio iterativo unidimensional actualiza iterativamente un arreglo X de tamaño N durante T pasos de tiempo:

$$X_{t+1}[i] = \frac{X_t[i-1] + X_t[i+1]}{2}, \quad 1 \leq i \leq N-2$$

Los extremos $X[0]$ y $X[N-1]$ son condiciones de frontera fijas. Este algoritmo aparece en simulaciones físicas como la ecuación de calor o la difusión discreta.

4.2. Versión secuencial

La implementación secuencial en `runSequential` itera paso a paso, actualizando todos los elementos en cada iteración antes de pasar a la siguiente. No presenta dificultades de sincronización porque hay un único hilo.

4.3. Versión paralela con barrera global (`runParallelBarrier`)

La versión paralela con barrera asigna un subconjunto de índices a cada hilo. Al final de cada paso t , todos los hilos deben esperar en una barrera `CyclicBarrier` antes de comenzar el paso $t+1$, garantizando que ningún hilo lea un valor de $X[i]$ que aún no ha sido actualizado por el hilo vecino.

Esta estrategia es correcta pero subóptima: los hilos pasan tiempo ocioso esperando en la barrera, incluso cuando el hilo vecino ya completó el trabajo que los afecta.

4.4. Versión paralela con phasers (runParallelPhaser)

La mejora fundamental consiste en reemplazar la barrera global por phasers punto a punto. Cada hilo t tiene dos phasers: uno compartido con el hilo $t - 1$ (izquierdo) y otro con el hilo $t + 1$ (derecho). Al terminar de procesar la celda frontera de su subdominio, el hilo señala (`arrive()`) al vecino correspondiente, sin esperar a todos los demás.

El esquema general de implementación es:

```

1 // Hilo t procesa indices [lo, hi]
2 for (int iter = 0; iter < iterations; iter++) {
3     // Senalar al vecino derecho que el extremo izquierdo esta
4     // listo
5     phaserLeft.arrive();
6     // Calcular elementos internos (no dependen de vecinos)
7     for (int i = lo + 1; i < hi; i++) {
8         newX[i] = (X[i-1] + X[i+1]) / 2.0;
9     }
10    // Esperar al vecino izquierdo antes de calcular el extremo
11    phaserLeft.awaitAdvance(iter);
12    newX[lo] = (X[lo-1] + X[lo+1]) / 2.0;
13    // Esperar al vecino derecho antes de calcular el extremo
14    phaserRight.awaitAdvance(iter);
15    newX[hi] = (X[hi-1] + X[hi+1]) / 2.0;
16 }
```

Mientras el hilo espera al vecino izquierdo para calcular el elemento frontera izquierdo, ya puede procesar todos los elementos internos que no dependen de ese vecino. Esto genera el efecto de *pipeline*: el trabajo útil se superpone con la espera de sincronización.

4.5. Resultados de ejecución del Ejercicio 4

El criterio de aprobación para `runParallelPhaser` es superar en al menos un $1,1\times$ el tiempo de `runParallelBarrier`, es decir, la versión con phasers debe ser al menos un 10 % más rápida que la versión con barrera global. Este umbral, modesto frente a los del Ejercicio 3, refleja una restricción inherente al problema: las dependencias vecino-a-vecino limitan el solapamiento posible, y la ganancia proviene exclusivamente del tiempo que cada hilo deja de estar ocioso en la barrera global.

Los resultados obtenidos en la ejecución son los siguientes:

Implementación	Aceleración esperada	Aceleración real	Resultado
<code>runParallelBarrier</code>	—	$1,0\times$ (base)	—
<code>runParallelPhaser</code>	$\geq 1,1\times$	$\approx 1,2\times$	PASS

Cuadro 2: Aceleración de `runParallelPhaser` respecto a `runParallelBarrier`.

Obtener $1,2\times$ frente al umbral de $1,1\times$ es un resultado significativo en este contexto. El margen de $0,1\times$ sobre el mínimo exigido confirma que la implementación con phasers logra

un solapamiento real entre cómputo y sincronización, no simplemente ruido de medición. Concretamente, ese 20 % de mejora proviene de que cada hilo, en lugar de detenerse en la barrera global al final de cada iteración, señala al vecino inmediato con `arrive()` y continúa procesando los elementos internos de su subdominio mientras espera. El tiempo ocioso acumulado a lo largo de todas las iteraciones T es lo que la versión con phasers elimina, y esa eliminación se traduce directamente en el 20 % observado.

Es importante notar que la ganancia es modesta pero acotada por la naturaleza del algoritmo: la fracción del tiempo de cada iteración que corresponde a los elementos frontera (los únicos que requieren datos del vecino) es pequeña respecto al total. Cuanto mayor sea el subdominio asignado a cada hilo, menor será esa fracción y, por tanto, menor la diferencia entre ambas versiones. En subdominios pequeños, donde los elementos frontera representan una proporción mayor, la ventaja del phaser sería más pronunciada.

5. Análisis Comparativo

5.1. ¿Cuál solución fue mejor?

Para el Ejercicio 3, `forall2dChunked` es claramente superior a `forall2d` en matrices de tamaño moderado a grande. La versión con chunks elimina la sobrecarga de gestión de N^2 tareas individuales y mejora la localidad de caché al procesar bloques contiguos de la matriz.

Para el Ejercicio 4, la versión con phasers obtuvo una aceleración de $1,2\times$ respecto a `runParallelBarrier`, superando el umbral mínimo exigido de $1,1\times$. Aunque la ganancia absoluta es moderada, es esperada: el algoritmo de promedio iterativo tiene dependencias vecino-a-vecino que limitan el solapamiento posible a la fracción del tiempo correspondiente a los elementos frontera de cada subdominio. La versión con phasers es la mejor solución disponible dentro de las restricciones del problema porque elimina precisamente esa espera ociosa, sin poder ir más allá sin cambiar la estructura del algoritmo.

5.2. Impacto del paralelismo de ciclos

El paralelismo de ciclos es efectivo cuando:

- Las iteraciones son independientes entre sí (sin dependencias de datos).
- El trabajo por iteración es suficientemente grande para amortizar la sobrecarga de creación de tareas.
- El número de tareas está bien dimensionado respecto al número de núcleos.

La multiplicación de matrices cumple todas estas condiciones idealmente, lo que explica la aceleración cercana a lineal observada.

5.3. Impacto del paralelismo de datos con phasers

El pipelining con phasers es más complejo de implementar pero ofrece mejor rendimiento cuando existe un patrón de dependencias vecino-a-vecino claro. La clave es identificar qué trabajo puede realizarse antes de recibir los datos del vecino y cuál debe esperar.

6. Limitaciones de los Modelos

6.1. Limitaciones del paralelismo de ciclos

1. **Dependencias de datos:** cualquier dependencia entre iteraciones rompe la paralelizabilidad directa. Algoritmos como la eliminación gaussiana sin pivoteo o la serie de Fibonacci no pueden paralelizarse con `forall2d` sin reestructuración.
2. **Acceso a memoria y caché:** la multiplicación de matrices accede a la columna $B[k][j]$ de forma no contigua en memoria (column-major access en row-major storage). Aunque Java almacena arreglos de arreglos como punteros independientes, el acceso repetido a la misma columna de B puede generar fallos de caché (cache misses) que limitan la ganancia práctica.
3. **Granularidad:** si el número de chunks no es múltiplo del número de núcleos, algunos hilos procesan más trabajo que otros, generando desequilibrio de carga (*load imbalance*).
4. **Límite de la Ley de Amdahl:** partes del código como la inicialización de matrices, la verificación de resultados y la infraestructura del *framework* de pruebas son secuenciales, imponiendo un techo teórico a la aceleración total.

6.2. Limitaciones del modelo de phasers

1. **Complejidad de programación:** el código con phasers es significativamente más difícil de escribir y depurar que el de barrera global. Los errores de sincronización (deadlocks, condiciones de carrera en fronteras) son difíciles de reproducir.
2. **Overhead de phasers:** cada operación `arrive()` y `awaitAdvance()` tiene un costo no despreciable. Para iteraciones muy cortas, este overhead puede superar el beneficio del pipelining.
3. **Escalabilidad:** el número de phasers crece linealmente con el número de hilos. La gestión de muchos phasers puede convertirse en un cuello de botella en sistemas con decenas de núcleos.
4. **Desequilibrio de carga:** si los subdominios no están balanceados (por ejemplo, los hilos frontera procesan menos elementos), el pipeline puede tener burbujas frecuentes.

7. Mejoras Propuestas

7.1. Mejoras para el Ejercicio 3

1. **Transposición de la matriz B:** transponer B antes de la multiplicación convierte el acceso a columnas en acceso a filas, mejorando dramáticamente la localidad de caché y potencialmente duplicando el rendimiento en matrices grandes.
2. **Multiplicación por bloques (blocking/tiling):** dividir las matrices en submatrices de tamaño ajustado a la caché L1/L2 permite reutilizar datos en caché antes de desalojarlos, reduciendo el número de accesos a memoria principal.

3. **Ajuste dinámico del tamaño de chunk:** en lugar de dejar que PCDDP elija el tamaño de chunk, configurarlo explícitamente en función del tamaño de caché y el número de núcleos puede mejorar el rendimiento en hardware específico.
4. **Uso de BLAS/LAPACK:** para producción, librerías nativas como OpenBLAS o Intel MKL ofrecen multiplicación de matrices altamente optimizada con instrucciones SIMD y gestión de caché avanzada, superando ampliamente lo posible en Java puro.

7.2. Mejoras para el Ejercicio 4

1. **Pipelining multinivel:** para más de dos hilos, se puede implementar un pipeline de mayor profundidad donde cada hilo avanza su fase de forma asíncrona respecto a todos sus vecinos, no solo el inmediato.
2. **Doble buffer (*double buffering*):** en lugar de copiar `newX` sobre `X` al final de cada iteración, alternar entre dos arreglos elimina la copia y reduce el uso de memoria.
3. **Vectorización:** el bucle interno del promedio puede beneficiarse de instrucciones SIMD si el JIT las aplica, lo que requiere asegurar que los arreglos estén alineados y el patrón de acceso sea secuencial.
4. **Reducción de barreras por bloque:** agrupar varias iteraciones por bloque temporal antes de sincronizar reduce la frecuencia de sincronización, a costa de mayor complejidad en el manejo de fronteras.

7.3. Mejoras generales de despliegue

1. **Perfilado antes de optimizar:** herramientas como Java Flight Recorder, JProfiler o async-profiler permiten identificar los verdaderos cuellos de botella antes de invertir en optimizaciones especulativas.
2. **Pruebas con JMH:** el framework Java Microbenchmark Harness (JMH) es más confiable que las mediciones manuales con `System.currentTimeMillis()` para benchmarks de rendimiento, ya que controla el calentamiento del JIT y los efectos de clase cargada.
3. **Configuración del JVM:** opciones como `-XX:+UseG1GC`, `-Xmx4g` (ya usado en el `pom.xml`) y `-XX:ParallelGCThreads` afectan el rendimiento de aplicaciones paralelas. Las pausas del GC pueden distorsionar las mediciones de tiempo.
4. **Portabilidad:** el número de núcleos varía entre entornos de desarrollo, integración continua y producción. El uso de `COURSERA_GRADER_NCORES` como variable de entorno es un buen patrón, pero en producción conviene parametrizar el nivel de paralelismo de forma más robusta.

8. Conclusiones

Los dos ejercicios demuestran que la paralelización efectiva requiere tanto elegir la primitiva adecuada (`forall2dChunked` vs. `forall2d`, `phasers` vs. barreras globales) como comprender el patrón de acceso a datos y las dependencias del algoritmo.

Para la multiplicación de matrices, `forall2dChunked` logra aceleraciones cercanas al número de núcleos disponibles con una modificación mínima del código secuencial, confirmando que los algoritmos embarazosamente paralelos son los mejores candidatos para paralelización directa.

Para el promedio iterativo, los phasers ofrecen un modelo de sincronización más fino que las barreras globales, permitiendo aprovechar el paralelismo de pipeline inherente al patrón de dependencias vecino-a-vecino. Sin embargo, la complejidad adicional de implementación y depuración debe justificarse con ganancias medibles en el contexto específico de ejecución.

En ambos casos, la escalabilidad práctica queda por debajo del ideal teórico debido a la sobrecarga de gestión de tareas, los efectos de caché, el desequilibrio de carga y las fracciones secuenciales inevitables del código. Las mejoras propuestas en este informe representan pasos concretos para acercar el rendimiento práctico al límite teórico en cada ejercicio.